

1: Develop a Program in C for the following:

- a. Define a structure Book with the following fields: Book_ID, Title, Author, Price, Availability Status (Available/Issued)**
- b. Dynamically allocate memory to store the details of N books, where N is entered by the user.**
- c. Implement the following functions:**
 - i. create() – To allocate memory and read the details of N books and store them in the allocated memory.**
 - ii. display() – To display the details of all books available in the library.**
 - iii. search() – To search for a book using Book ID and display the book details if found.**
 - iv. issueBook() – To issue a book by updating its availability status and display an appropriate message.**
 - v. returnBook() – To return an issued book by updating its availability status and display an appropriate message.**
- d. Provide a menu-driven interface with the following options:**
 - i. Add Book Records**
 - ii. Display All Book Records**
 - iii. Search Book by Book ID**
 - iv. Issue a Book**
 - v. Return a Book**
 - vi. Exit**

Theory:

Structure: A structure is used to store different details of a book such as Book ID, Title, Author, Price, and Availability Status under one name.

Dynamic Memory Allocation: malloc() dynamically allocates memory for N book records at runtime, and free() releases the allocated memory after use.

Functions & Menu: User-defined functions perform operations such as creating, displaying, searching, issuing, and returning books through a menu-driven interface.

Program Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Book
{
```

```
int Book_ID;
char Title[50];
char Author[50];
float Price;
char Status[10];
};

/* Function to create book records */
struct Book* create(int n)
{
    int i;

    struct Book *b;

    /* Dynamically allocate memory */
    b = (struct Book*)malloc(n * sizeof(struct Book));

    if (b == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    for (i = 0; i < n; i++)
    {
        printf("\nEnter details of Book %d\n", i + 1);

        printf("Book ID: ");
        scanf("%d", &b[i].Book_ID);

        printf("Title: ");
        scanf(" %[^\\n]", b[i].Title);

        printf("Author: ");
        scanf(" %[^\\n]", b[i].Author);

        printf("Price: ");
        scanf("%f", &b[i].Price);

        printf("Availability Status (Available/Issued): ");
        scanf("%s", b[i].Status);
    }

    return b;
}
```

```

}

/* Function to display all book records */
void display(struct Book b[], int n)
{
    int i;

    printf("\n----- BOOK RECORDS ----- \n");

    for (i = 0; i < n; i++)
    {
        printf("\nBook ID      : %d", b[i].Book_ID);
        printf("\nTitle       : %s", b[i].Title);
        printf("\nAuthor      : %s", b[i].Author);
        printf("\nPrice       : %.2f", b[i].Price);
        printf("\nAvailability Status: %s\n", b[i].Status);
    }
}

/* Function to search a book using Book ID */
void search(struct Book b[], int n)
{
    int id;
    int i;
    int found = 0;

    printf("\nEnter Book ID to search: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++)
    {
        if (b[i].Book_ID == id)
        {
            printf("\nBook Found!\n");

            printf("Book ID      : %d\n", b[i].Book_ID);
            printf("Title       : %s\n", b[i].Title);
            printf("Author      : %s\n", b[i].Author);
            printf("Price       : %.2f\n", b[i].Price);
            printf("Availability Status: %s\n", b[i].Status);

            found = 1;
            break;
        }
    }
}

```

```

    }

    if (found == 0)
    {
        printf("Book not found.\n");
    }
}

/* Function to issue a book */
void issueBook(struct Book b[], int n)
{
    int id;
    int i;

    printf("\nEnter Book ID to issue: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++)
    {
        if (b[i].Book_ID == id)
        {
            if (strcmp(b[i].Status, "Available") == 0)
            {
                strcpy(b[i].Status, "Issued");

                printf("Book issued successfully.\n");
            }
            else
            {
                printf("Book is already issued.\n");
            }
        }

        return;
    }
}

printf("Book not found.\n");
}

/* Function to return a book */
void returnBook(struct Book b[], int n)
{
    int id;
    int i;

```

```

printf("\nEnter Book ID to return: ");
scanf("%d", &id);

for (i = 0; i < n; i++)
{
    if (b[i].Book_ID == id)
    {
        if (strcmp(b[i].Status, "Issued") == 0)
        {
            strcpy(b[i].Status, "Available");

            printf("Book returned successfully.\n");
        }
        else
        {
            printf("Book is not currently issued.\n");
        }
    }

    return;
}

printf("Book not found.\n");
}

/* Main function */
int main()
{
    struct Book *b = NULL;

    int n;
    int choice;

    printf("Enter the number of books: ");
    scanf("%d", &n);

    /* Menu-driven program */
    do
    {
        printf("\n===== LIBRARY MENU =====\n");
        printf("1. Add Book Records\n");
        printf("2. Display All Book Records\n");
        printf("3. Search Book by Book ID\n");
    }

```

```
printf("4. Issue a Book\n");
printf("5. Return a Book\n");
printf("6. Exit\n");
printf("=====\\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        b = create(n);

        if (b != NULL)
        {
            printf("\\nBook records added successfully.\\n");
        }
        break;

    case 2:
        if (b == NULL)
        {
            printf("\\nPlease add book records first.\\n");
        }
        else
        {
            display(b, n);
        }
        break;

    case 3:
        if (b == NULL)
        {
            printf("\\nPlease add book records first.\\n");
        }
        else
        {
            search(b, n);
        }
        break;

    case 4:
        if (b == NULL)
        {
```

```

        printf("\nPlease add book records first.\n");
    }
    else
    {
        issueBook(b, n);
    }
    break;

case 5:
    if (b == NULL)
    {
        printf("\nPlease add book records first.\n");
    }
    else
    {
        returnBook(b, n);
    }
    break;

case 6:
    printf("\nExiting the program...\n");
    break;

default:
    printf("\nInvalid choice.\n");
}

} while (choice != 6);

/* Release dynamically allocated memory */
if (b != NULL)
{
    free(b);
}

return 0;
}

```

Output

Output to be Written in record in blank side.

Inference